Project Log

Continuing the FPGA Matrix-Multiplication Accelerator

TeraNova – FPGA Acceleration

Author	Kai Janssen
Student number	4994817
Period covered	30 September – 7 October 2026
Platform	AMD ZCU102 (Zynq UltraScale+ `xczu9eg-ffvb1156-2-e`)
Tools	Vivado / Vitis / Vitis HLS 2025.1.1

October 7, 2026

Contents

1 Purpose and scope **2**

2 Status summary **2**

3 Chronological log **2**

3.1 30 September 2026 – Setup of the received project 2

3.2 30 September 2026 – Analysis of the previous design 3

3.3 2 October 2026 – Restoring the two IP cores 4

3.4 2 October 2026 – Building the IP and updating the block design 5

3.5 4 October 2026 – Hardware build attempt 5

3.6 7 October 2026 – Pre-build work while waiting for the licence 5

4 Files created and modified **7**

5 Key decisions **8**

6 Open issues and next steps **8**

1 Purpose and scope

This log records the work I carried out to take over the FPGA matrix-multiplication accelerator developed by the previous project group. The overall project goal is to determine whether moving the inverse-scattering calculations of TeraNova's metrology system to the FPGA, and accelerating them in hardware, improves the run time of the system. The work in this period focused on:

1. reproducing the previous group's Vivado and Vitis projects on my own PC;
2. analysing their design and identifying its bottlenecks;
3. restoring two IP cores that were missing from the hand-over, and making the PS–PL interaction robust;
4. preparing a new hardware build.

All work was done in the workspace `C:\PLD`; the files received from the previous group are kept unchanged in a separate folder.

2 Status summary

Item	Status
Vivado project opens in 2025.1.1	**Done**
Vitis platform and application build without errors or warnings	**Done**
Interrupt handling in the application completed	**Done** (not yet tested on board)
Analysis of the previous design and report	**Done**
`RAM_INTERFACE` rewritten (HLS), synthesised for `xczu9eg` and packaged	**Done**
`matrix_partitioner_B` recovered and re-packaged	**Done**
Block design updated to the latest IP builds, validated without errors	**Done**
RTL (GHDL), HLS C and C/RTL co-simulation of the new data path	**Done**
Updated application compiled (against the old hardware export)	**Done**
Unused IP repository `Cezar_repo` removed; all 148 IPs up to date	**Done**
Bitstream and hardware export (`.xsa`)	**Blocked** (licence)
Test on the ZCU102	**Open** (board and licence pending)

Table 1: Status at the end of the logged period.

3 Chronological log

3.1 30 September 2026 – Setup of the received project

Received files. The previous group handed over (via Teams): the Vivado project `S7_integration.zip` (1.27 GB), the Vitis workspace `S7_integration_vit.zip`, five packaged IP cores, the HLS source of the DDR interface and a short README. The README stated

that the project depends on the path `C:/PLD/s7_integration`, that the interrupt never worked, and that Vivado/Vitis 2025.1.1 was used.

Workspace. I created `C:\PLD` as workspace so that the absolute paths inside the projects remain valid, and extracted the Vivado project with 7-Zip to `C:\PLD\S7_integration`.

Identification of the contents. Inspection of the archives showed that the target is a Zynq UltraScale+ ZCU102 board (`xczu9eg`), not a Spartan-7 as the name "S7" suggested. The five IP archives turned out to be: `RAM_INTERFACE` (HLS), `MatMul5x5_Acc`, `MatMul5x5_PAR_IF`, `matrix_partitioner_B` and an older, unused `matrix_partitioner`.

IP repositories. From the project file I determined that Vivado expects IP repositories at `../ip_repo` and `../Cezar_repo`. I placed each IP in its own subfolder of `C:\PLD\ip_repo` (two of the archives used the same folder name and would otherwise have overwritten each other) and created an empty placeholder `Cezar_repo` (removed again on 7 October, see Stage 8).

Problem 1 – tool version. On opening, every IP in the block design was locked with status "Newer software version". The cause was that I had Vivado 2025.1 installed, while the project was created in 2025.1.1. I installed 2025.1.1 (with Zynq UltraScale+ MPSoC device support and cable drivers) in `C:\Xilinx\2025.1.1`. Afterwards 143 of the 145 IPs were up to date.

Problem 2 – outdated IP in the hand-over. `RAM_INTERFACE` and `matrix_partitioner_B` remained locked with "IP revision change". The recommended version was *older* than the one in the design (e.g. partitioner Rev. 3 in the design versus Rev. 1 in the archive). By comparing with the copies Vivado keeps in `S7_integration.gen/.../ipshared`, I confirmed that the handed-over archives were outdated. "Upgrade IP" would therefore have downgraded the design, so I did not use it.

Vitis workspace. I extracted `S7_integration_vit.zip` to `C:\PLD\S7_integration_vit` and opened it in the Vitis Unified IDE. The build files showed that the previous group used exactly the same paths, so no path changes were needed. Platform `interrupt_rerun_plat` and application `interrupt_rerun_test` both built successfully. I selected "Always build platform with application" (workspace preference).

Interrupt fix. Reading `helloworld.c` showed why the interrupt never worked: the GIC was configured and the handler connected, but the processor's exception system was never set up. I added `XScuGic_SetPriorityTriggerType()`, `Xil_ExceptionInit()`, `Xil_ExceptionRegisterHandler()` and `Xil_ExceptionEnable()`, plus a check of the connect status. I also removed the two remaining compiler warnings. The original file was kept in `S7_integration_vit\backup`. The application built without warnings.

Lesson learned: one edit was lost because Vitis still had the file open and saved its older copy over the change. Files must be closed in the IDE before they are modified outside it.

3.2 30 September 2026 – Analysis of the previous design

I analysed the HDL, HLS and C sources, the block design, and the timing and utilisation reports of the previous build. The design multiplies two 50×50 matrices of 64-bit integers in 5×5 blocks:

1. the CPU writes A and B into block RAM as 10 000 single 32-bit transactions;
2. `matrix_partitioner_B` reads the blocks element by element (4 clock cycles per element);
3. `MatMul5x5` multiplies two blocks in one clock cycle (1 257 DSPs);
4. an accumulator sums 10 partial products per output block;
5. `RAM_INTERFACE` writes the results to the PL-side DDR4, from where the CPU reads them back one by one.

Main finding. The design is dominated by data movement. Fetching one block pair takes

about 100 cycles while multiplying it takes about 5, so the compute unit is idle about 95 % of the time. I estimated the end-to-end run at roughly 2.5–4 ms, and estimated that the proposed improvements (burst transfers instead of CPU single writes, removing the partitioner's wait states, banked memory, data reuse, a narrower pipelined multiplier) could reduce this to roughly 30–50 µs. I also noted that the current accelerator is probably slower than a CPU-only implementation, and that the gain grows with matrix size.

Deliverable. A full report (`previous_group_analysis.tex`, saved in `C:\PLD\report`) with 14 numbered findings, cycle and resource budgets, proposed improvements and expected speed-ups. All figures in it are estimates pending measurements on the board.

3.3 2 October 2026 – Restoring the two IP cores

Decision. Two options were considered: (A) re-package the generated HDL found in the project, or (B) rebuild the IPs from source. I chose **option B**, because:

- it gives the group editable source code instead of opaque generated HDL;
- the improvement plan requires changing `RAM_INTERFACE` anyway;
- the old `RAM_INTERFACE` handshake (plain `ap_none` wires on a free-running block) was not a proper protocol and could capture a result block twice or miss one.

`RAM_INTERFACE` (rewritten in HLS).

- Input is now an `hls::stream` with an `ap_fifo` interface; this maps directly onto the accumulator's existing valid/ready signals (`C_valid` → `data_empty_n`, `C_ready` ← `data_read`).
- Elements are extracted with `.range()` instead of a C union; the inner loop is pipelined (II=1).
- `done` rises only after all writes of the last block are complete.

`matrix_partitioner_B`. The newest VHDL was recovered from the project's `ipshared` folder. Its ports are identical to the archived version, so it only needed re-packaging with core revision 4.

Application update (`helloworld.c`). Separate timing of BRAM load, PL compute, result read-back and CPU-only reference; a 1-second timeout with a GPIO fallback so the program can never hang silently; the start signal is now a toggle so the design can be run repeatedly; two complete runs per execution; build optimisation changed from `-O0` to `-O2`.

Verification (without hardware).

Test	Result
HLS C simulation of `RAM_INTERFACE`: two complete matrices, full 64-bit values	PASS
RTL simulation (GHDL) of the full PL path, test data from `helloworld.c`	PASS
RTL simulation, random full-range 64-bit data	PASS
RTL simulation with a slow writer (back-pressure, 1 500 cycles/block)	PASS, no lost or duplicate blocks
Measured PL run time in simulation	101 008 cycles (≈1.01 ms at 100 MHz)

Table 2: Verification results.

Files delivered to `C:\PLD`: `hls\RAM_INTERFACE` (source, testbench, `build_ip.bat`), `scripts` (three Vivado Tcl scripts), `sim` (GHDL testbench), the updated application and `README_UPGRADE.md`. All replaced files were backed up first.

3.4 2 October 2026 – Building the IP and updating the block design

1. `build_ip.bat` completed: the new `RAM_INTERFACE` (core revision 2114811662) was installed in `C:\PLD\ip_repo\RAM_INTERFACE` with the expected ports.
2. `1_repackage_partitioner.tcl` and `2_update_block_design.tcl` were run in the Vivado Tcl console.
3. Result: **all 145 IPs up to date**, nothing locked. I verified the saved block design: the stream handshake, `done` (interrupt, GPIO channel 2 and ILA), clock, reset, base address and the AXI connection to the DDR4 are all connected as intended, and the address map is unchanged (PL DDR4 at `0x4_0000_0000`).

3.5 4 October 2026 – Hardware build attempt

I ran `3_build_and_export.tcl`. Generating the block design output products succeeded, but all synthesis runs stopped with:

```
ERROR: [Common 17-345] A valid license was not found for feature
'Synthesis' and/or device 'xczu9eg'.
```

The free Vivado edition does not cover the ZU9EG; synthesis, implementation and bitstream generation for this device require a licence. Steps that do not need a licence (HLS, block-design editing, simulation, Vitis software builds, programming the board) all worked. The separate message "Failed to create directory 'C' " appeared at start-up but did not stop the run. The width-mismatch critical warnings in the log were already present in the previous group's design and are harmless.

The group's licence is **node-locked** and registered to one group member. The options identified are: remote access to the licensed PC, having the licence holder run the build script on a copy of `C:\PLD`, or re-hosting the licence to a shared build PC. A new build is only needed when the hardware changes; software changes never need one.

3.6 7 October 2026 – Pre-build work while waiting for the licence

While the licence for the ZCU102 is being set up, I listed which steps of the plan do not need it and carried them out in order. Only synthesis/implementation/bitstream for the `xczu9eg`, the Vitis platform update (needs the new `.xsa`) and the board run remain blocked.

Stage 1 – Backup. Before changing the projects, I copied `S7_integration` and `S7_integration_vit` to `C:\PLD\backup\S7_integration_2026-10-07` and `...\S7_integration_vit_2026-`

Stage 2 – HLS build of `RAM_INTERFACE`. `build_ip.bat` ran to completion (log in `hls\RAM_INTERFACE\build_log.txt`). Contrary to expectation, Vitis HLS C synthesis for the `xczu9eg` did *not* need the licence. The remaining warnings are harmless: one about variable-indexed range selection (`.range()`), power-on initialisation of the three static registers, and that `done` (`ap_none`) cannot be checked automatically by co-simulation.

Stage 3 – C/RTL co-simulation. The first attempt failed before simulating:

```
ERROR: [COSIM 212-345] Cosim only supports the following 'ap_ctrl_none'
designs: (1) combinational designs; (2) pipelined design with II of 1;
```

Item	Result
C simulation (2 complete matrices)	PASS, 0 errors
Timing, 100 MHz target (10 ns)	7.30 ns estimated; Fmax ≈ 137 MHz
Main loop `ROWS_COLS`	pipelined, II = 1
Latency per 5×5 block (estimate)	41 cycles (0.41 µs)
Resources	2 708 FF, 1 943 LUT, 2 DSP, 0 BRAM (<1 %)
Packaging and installation	IP installed in `ip_repo\RAM_INTERFACE`

Table 3: HLS results of the rebuilt `RAM_INTERFACE`.

```
(3) designs with array streaming or hls_stream or AXI4 stream ports.
```

This is a tool limitation, not a design error: `RAM_INTERFACE` is free-running (`ap_ctrl_none`), takes several cycles per block and writes through `m_axi`. To still verify the generated RTL, I added a co-simulation-only build: `RAM_INTERFACE.cpp` selects `ap_ctrl_hs` when the macro `COSIM_HS` is defined (otherwise `ap_ctrl_none`, so the packaged IP is unchanged), and the new `hls_config_cosim.cfg` sets `-DCOSIM_HS` and uses its own work folder (`work_cosim`). `run_cosim.bat` runs synthesis and co-simulation of this variant.

Result: **C/RTL co-simulation PASS** – all 2×100 blocks written correctly by the generated RTL in xsim, and the `done` check passed. The simulated time per block was about 780 ns (≈78 cycles instead of the 41 estimated, because of waiting for AXI write responses), i.e. about 7.8 µs to write a complete 50×50 result.

Stage 4 – Re-packaging the partitioner. `1_repackage_partitioner.tcl` ran without errors (integrity check passed). It reported "Old core revision: 4": the re-packaging had already been applied on 2 October, so the run only re-saved the same revision.

Stage 5 – Block design. Before running `2_update_block_design.tcl` I inspected the saved block design. It already contained everything that script does (partitioner revision 4; the new `RAM_INTERFACE` ports wired to the accumulator; `done` to GPIO, interrupt and ILA; `m_axi_gmem` to `axi_smc/S01_AXI`; address segment at `0x4_0000_0000`, 512 MB). Re-running it would delete and recreate a correctly wired cell, so I did not run it. The only issue was that `RAM_INTERFACE_0` showed as *locked*, because today's HLS build has a new core revision. As the ports are identical, I wrote `2b_upgrade_after_rebuild.tcl`, which only upgrades the two IPs, validates and saves. Result:

- `RAM_INTERFACE_0` upgraded from revision 2114811662 to 2114818764;
- `validate_bd_design`: no errors (only the known SmartConnect, AWUSER/ARUSER width and debug-net warnings);
- checked in the saved `accu_to_DDR.bd`: all `RAM_INTERFACE_0` connections and the address segment unchanged.

Stage 6 – Application build. The updated `helloworld.c` (from 2 October) had not been built yet. I built `interrupt_rerun_test` in Vitis against the existing (old) platform: **build successful**, no errors (text 93 748 B, data 9 576 B, bss 582 128 B). The code only uses hardware names that remain the same in the new design, so it is expected to build unchanged against the new `.xsa`.

Stage 7 – Board bring-up preparation. None of these steps needs the licence, but most need the board. Possible now: installing the JTAG cable drivers and the Silicon Labs CP210x USB-UART driver, and preparing a serial terminal profile (115 200 baud, 8N1, no flow control). With the board: check that four CP210x COM ports appear (the lowest is PS UART0), set SW6 to JTAG boot (all ON), and check that the Hardware Manager detects the `xczu9eg`.

Stage 8 – Removing the unused IP repository. The project still listed `../Cezar_repo` as a second IP repository. No IP in the block design came from it; the folder only held my placeholder README. In the Vivado Tcl console:

```
set_property ip_repo_paths C:/PLD/ip_repo [current_project]
update_ip_catalog
```

Vivado then loaded only `c:/PLD/ip_repo`. I checked the saved `S7_integration.xpr` (only one `IPRepoPath` left), searched the project file, block design, IP configuration files and scripts for remaining references to the folder (none; the hierarchy block `Cezars_pin_paradise` only shares the name and is unrelated), and then deleted `C:\PLD\Cezar_repo`. `report_ip_status` afterwards: **148 IPs, all up to date**, none locked. Three Constant IPs inside the SmartConnects are marked "Up-to-date (discontinued)", which needs no action. The four custom IPs (`MatMul5x5_Acc` Rev. 4, `MatMul5x5_PAR_IF` Rev. 2, `matrix_partitioner_B` Rev. 4, `RAM_INTERFACE` Rev. 2114818764) are all found in `ip_repo`.

Stage 9 – Generate Output Products (licence check). On opening the project, Vivado warned that the generated files of `accu_to_DDR_axi_smc_0` were missing. I ran *Generate Output Products* (out of context per IP, 12 jobs). Three out-of-context runs failed with `[Common 17-345] A valid license was not found for feature 'Synthesis' and/or device 'xczu9eg'`: `accu_to_DDR_axi_smc_0`, `accu_to_DDR_matrix_partitioner_B_0_0` and `accu_to_DDR_RAM_INTERFACE_0_1`. These are exactly the IPs without a cached synthesis result (the two rebuilt IPs and the SmartConnect affected by the upgrade); all others were reused from the cache. This confirms the 4 October finding: on my PC, even generating output products needs the licence. It is not related to Stage 8. The runs will be redone by `3_build_and_export.tcl` on the licensed PC.

Generation also gave twelve `[BD 41-2383] Width mismatch` critical warnings: six connections, each reported by validation and again by generation. They concern the address path of the previous group's design: partitioner `bram_addr_A/B` (12 bits) → `mult_gen_*/A` (18 bits), and `mult_gen_*/P` (21 bits) → `Part0..3/addrb` (32 bits). In every case the narrower side covers the full address range (12 bits = 4096 values for 2500 elements per matrix), and the unconnected upper bits are tied to 0 in synthesis. They are harmless and were not caused by the upgrade.

4 Files created and modified

Path (under `C:\PLD`)	Content
`ip_repo\*`	IP repository (5 IPs, one folder each)
`Cezar_repo\README.txt`	Placeholder for the missing repository; folder deleted and removed from the project on 7 Oct
`S7_integration\S7_integration.xpr`	IP repository list reduced to `../ip_repo` (7 Oct)
`hls\RAM_INTERFACE\*`	Rewritten HLS source, testbench, config, build script
`hls\RAM_INTERFACE\hls_config_cosim.cfg`, `run_cosim.bat`	Co-simulation build (`ap_ctrl_hs` via `COSIM_HS`), 7 Oct
`hls\RAM_INTERFACE\build_log.txt`, `cosim_log.txt`	Logs of the 7 Oct HLS build and co-simulation
`ip_repo\matrix_partitioner_B\*.vhd`	Newest partitioner VHDL (revision 4)

Path (under C:\PLD)	Content
scripts\1_ ... 3_*.tcl	Re-package, block-design update, build and export
scripts\2b_upgrade_after_rebuild.tcl	Upgrade-only update after an IP rebuild (7 Oct)
sim*	GHDL testbench of the complete PL data path
S7_integration_vit\ ...\src\helloworld.c	Updated application
S7_integration_vit\ ...\src\UserConfig.cmake	Optimisation -O0 → -O2
report\previous_group_analysis.tex	Analysis report of the previous design
README_UPGRADE.md	Step-by-step upgrade guide
backup*	Old partitioner VHDL and RAM_INTERFACE IP
backup\S7_integration*_2026-10-07	Full copies of the Vivado project and Vitis workspace (7 Oct)
S7_integration_vit\backup*	Original and pre-upgrade application files

5 Key decisions

Decision	Rationale
Workspace at C:\PLD	Keeps the previous group's absolute paths valid without editing project files.
Install Vivado 2025.1.1	Exact version match prevents IP locking and upgrade issues.
Do not use "Upgrade IP" for the two locked IPs	The offered version was older than the design's; upgrading would have downgraded the hardware.
Rebuild RAM_INTERFACE from source (option B)	Editable code, a correct handshake, and a starting point for the planned improvements.
Verify by simulation before building	Hardware builds are long and currently licence-bound; simulation catches functional errors cheaply.
Co-simulate a separate ap_ctrl_hs variant	Co-simulation does not support this ap_ctrl_none design; a macro-selected variant verifies the same data path and m_axi writes without changing the packaged IP.
Do not re-run 2_update_block_design.tcl; use an upgrade-only script	The block design was already rewired; only the IP revision changed, and an upgrade keeps the existing connections.
Remove Cezar_repo from the project	No IP came from it; a second, empty repository only adds a source of confusion and a risk of picking up stale IP versions later.

6 Open issues and next steps

1. **Build**: run 3_build_and_export.tcl on the PC holding the node-locked licence; this also regenerates the three out-of-context runs that failed on my PC (Stage 9). Check that the worst negative slack is ≥ 0; obtain interrupt_rerun.xsa.

2. **Vitis**: switch the platform to the new `.xsa` and rebuild platform and application.
3. **Board preparation**: install the JTAG and CP210x drivers now; once the board is available, check the COM ports, set SW6 to JTAG and test the JTAG connection in the Hardware Manager.
4. **Board test**: run on the ZCU102 (JTAG + UART at 115 200 baud); expected output ends with `ALL RUNS PASSED`.
5. **Measure**: BRAM load time, PL compute time, read-back time and CPU-only time, to replace the estimates in the analysis report.
6. **Improve**: start with the low-effort changes (partitioner wait states), then the burst-based data path.